

Sensiq

Technical Reports: CL-2026-42, July 2026

Simon Völkl, Johannes Schieder, Sebastian Rosner and Andre Tien Vu 

Department of Electrical Engineering, Media, and Computer Science
Ostbayerische Technische Hochschule Amberg-Weiden
Amberg, Germany

Abstract—
Index Terms—

I. INTRODUCTION

Reliable indoor environment quality (IEQ) monitoring is a recurring requirement in laboratories, production halls, and similar facilities, where ambient conditions like indoor air quality (IAQ) directly affect experimental validity, product quality, and occupational safety. Existing IoT-based monitoring platforms address parts of this problem but typically rely on on-premises servers, offer only coarse sampling, or restrict access through proprietary apps, as discussed in detail in Section II.

This report describes Sensiq, a microcontroller unit (MCU) based Internet of Things (IoT) architecture that closes this gap by combining ESP32 (Espressif Systems) edge devices with a serverless Amazon Web Services (AWS) cloud backend. Sensor readings are transmitted over Message Queuing Telemetry Transport (MQTT) secured with Transport Layer Security (TLS), validated and persisted in the cloud via AWS services for a live and historical view, and exposed through Representational State Transfer (REST) Application Programming Interface (API) endpoints. A React-based web frontend consumes these endpoints to visualize live and historical data, and threshold-driven email alerts notify responsible personnel of critical events. The remainder of this section presents the mission statement and the underlying motivation for the system.

A. Mission Statement

Sensiq is an end-to-end environmental monitoring system that enables laboratory operators, occupational safety officers, facility managers, and IT administrators to observe critical ambient parameters such as temperature, humidity, light intensity, gas concentration, and flame exposure in real time, while also providing scalable access to historical data for long-term analysis. Its overarching goal is to deliver reliable, low-latency live monitoring together with cost-efficient historical analytics on a fully serverless backend, without depending on vendor-locked third-party dashboards.

B. Motivation

In laboratory and small-scale production environments, deviations in temperature, humidity, light intensity, gas concentration, or fire exposure can invalidate experiments, damage sensitive samples, or directly endanger personnel [1, 2]. Operators therefore need both an immediate view of current conditions

and a complete record of past readings for quality assurance and incident analysis.

In practice, this combination is hard to obtain. Existing academic IoT platforms such as those by Mumtaz et al. [3] and Marques and Pitarma [4] rely on on-premises stacks and use sampling or aggregation intervals that are too coarse to react to short-lived, safety-relevant events.

Sensiq is motivated by this gap. By pairing an inexpensive ESP32 sensor node with a serverless cloud backend, the system aims to provide real-time visibility, durable historical storage, and programmatic access through open REST endpoints, while keeping infrastructure and operating cost low enough for routine use in academic and industrial laboratories. The full source code is available on GitHub under the MIT License [5].

C. Document Structure

The remainder of this report is organised as follows. Section II positions Sensiq against existing academic and commercial monitoring solutions and motivates the design decisions taken in this project. Section III introduces the conceptual building blocks of the system and presents the end-to-end architecture in a technology-agnostic manner.

The subsequent three sections refine this concept along the data flow: Section IV details the ESP32-based sensor node and its secure MQTT publication to AWS IoT Core; Section V describes the serverless cloud pipeline, covering both the low-latency live route and the long-term history route; and Section VI explains how end users consume the data through the React-based web frontend and how they are notified of critical events via email. Section VII assesses the resulting system with respect to testability and operating cost. Finally, Section VIII summarises the contribution of the report and outlines directions for future work.

II. RELATED WORK

Several IoT-based platforms for indoor environmental monitoring have been proposed in recent years. This section discusses two representative academic systems — the on-premises monitoring and prediction platform by Mumtaz et al. [3] and the iAQ+ laboratory supervision system by Marques and Pitarma [4] — and contrasts their backend architecture, sampling strategy, and notification design with the design decisions taken for Sensiq.

A. On-Premises IoT Air Quality Monitoring with Predictive Analytics

Mumtaz et al. [3] present an IoT-based indoor air quality monitoring and prediction platform motivated by the COVID-19 pandemic. Their sensor node combines an ATmega328P microcontroller with a NodeMCU Wi-Fi module and a broad set of sensors covering eight pollutants (NH_3 , CO, NO_2 , CH_4 , CO_2 , PM 2.5) as well as ambient temperature and humidity. The node, deployed in a research laboratory, transmits readings via GSM or Wi-Fi to a web portal and mobile app, and generates alerts when sensor values exceed predefined thresholds. Beyond pure monitoring, the authors additionally apply machine learning to the recorded data to predict future pollutant concentrations.

The work shares Sensiq's general topology of a multi-sensor edge node, a cloud-side analytics layer, and a web-based user interface, and the ATmega328P with NodeMCU is functionally comparable to the ESP32 used in Sensiq (Section IV). Beyond this surface similarity, however, the two systems pursue fundamentally different goals, and it is precisely these differences that motivate Sensiq.

First, Mumtaz et al. rely on a local web server and database. This ties the platform to continuous on-premises operation, complicates remote access, and makes horizontal scaling to additional sites or users an explicit infrastructure project. Sensiq instead targets a fully serverless cloud backend (Section V), which scales on demand, removes the need to operate dedicated server hardware, and keeps idle cost close to zero — a key requirement for routine use in academic and small industrial laboratories.

Second, their sampling interval of five minutes is geared towards trend analysis and pollutant forecasting rather than immediate reaction. For safety-relevant events such as fire exposure or sudden gas concentration spikes, this granularity is too coarse. It is likewise insufficient for product quality assurance, where maintaining steady environmental parameters such as temperature and humidity is essential and short-lived deviations must be detected before they compromise samples or production batches. Sensiq's MQTT-over-TLS pipeline is designed for low-latency live monitoring and threshold-driven email alerts, so that critical conditions surface within seconds rather than minutes.

Third, the reported battery autonomy of around twenty hours limits the node to scenarios with permanent power or frequent recharging. While both the ATmega328P/NodeMCU combination and the ESP32 in principle support deep-sleep operation, Mumtaz et al. do not exploit it. Sensiq does not yet implement deep sleep either, but its modular ESP32-based node is explicitly prepared for this extension, which is identified as future work in Section VIII to enable substantially longer unattended operation.

In summary, Mumtaz et al. [3] demonstrate that a multi-sensor IoT node combined with cloud-side analytics is a viable approach to indoor environmental monitoring, and their predictive layer illustrates how such a platform can be enriched with forecasting. Sensiq adopts the same overall

topology but deliberately shifts the emphasis: away from an on-premises stack with coarse, forecast-oriented sampling, towards a serverless backend with low-latency live monitoring, threshold alerts, and durable historical storage accessible via open REST endpoints.

B. iAQ+ Laboratory Environment Supervision System

Marques and Pitarma [4] present iAQ+, an IoT based environmental quality management system for laboratory conditions. An ESP8266 node equipped with a Bosch BME680 sensor captures temperature, relative humidity, barometric pressure, and a qualitative air quality index, and forwards them to a web management system and an iOS smartphone application that supports configurable thresholds and push notifications. The sensor selection and the Wi-Fi-based node topology resemble Sensiq's hardware (Section IV), but the differences in backend, frontend, and timing behaviour are what matter for Sensiq's positioning.

On the backend side, iAQ+ is built on a SQL Server database accessed through the .NET framework, i.e. a classical server-based stack with the same operational drawbacks already discussed for Mumtaz et al.: the platform requires a continuously running server, and scaling to additional sites or tenants is an explicit infrastructure activity. Sensiq instead delegates persistence and request handling to managed serverless AWS services (Section V), so that scaling and availability are handled by the cloud provider and idle cost remains negligible.

The frontend design of iAQ+ is also more restrictive than Sensiq's. The smartphone application is limited to iOS 12 and above, with no Android counterpart, which excludes a significant portion of potential users. The accompanying web dashboard exposes historical data only at hourly, daily, and monthly aggregations and does not provide a true live view or short-term (e.g. minute-level) history. Sensiq targets platform-independent access through a React-based web frontend served over open REST endpoints and explicitly distinguishes a low-latency live route from a long-term history route (Section VI).

Finally, iAQ+ does not state a measurement and transmission frequency, nor does it quantify the latency between a threshold violation and the delivered push notification. For safety-relevant events such as fire exposure or sudden gas concentration spikes, this lack of guaranteed timing is a substantive limitation. Sensiq is designed around an explicit low-latency MQTT-over-TLS pipeline and threshold-driven email alerts so that critical conditions surface within seconds, and its notification path is part of the evaluation in Section VII.

III. TECHNICAL CONCEPT

High-level, technology-agnostic view of the system. This section introduces the conceptual building blocks and the end-to-end data flow. Detailed component-level descriptions follow in the Hardware, AWS Services, and User Interaction sections.

A. Technical Building Blocks

Enumerate the logical components of the system without binding them yet to concrete products: edge sensing node,

secure transport channel, ingestion gateway, validation stage, hot-path store (live data), cold-path store (history), query API, notification channel, and user interface. Briefly justify the chosen programming languages (C++ firmware, Python lambdas, TypeScript infrastructure, React UI).

B. System Architecture

Present the architecture diagram (see template: fig:cloud-architecture) and walk the reader through the data flow: ESP32 -> MQTT/TLS -> AWS IoT Core -> validation Lambda -> dual persistence (DynamoDB for live, S3 + Athena for history) -> REST endpoints -> frontend / SNS.

IV. HARDWARE

The sensor node is the physical edge component of the Sensiq system and is built around an ESP32 microcontroller hosting a set of environmental sensors that cover temperature, humidity, ambient light, air pressure, volatile organic compounds (VOCs), flame radiation and high-temperature events. The following part of this section first lists the individual hardware components and their characteristics, then describes the firmware-level measurement workflow (sensor initialization, periodic sampling, averaging and MQTT publication), and finally documents the Steinhart–Hart conversion used for the analog thermistor readings. The resulting JSON payload sent to AWS IoT Core is described in Section IV-A.

Component Overview

The following hardware components are used in the sensor node:

- ESP32 NodeMCU board** Open-source development board carrying the ESP-WROOM-32 module, breaking out 38 pins and providing a CP2102 USB-to-serial bridge for programming and power [6].
- ESP-WROOM-32 module** Xtensa dual-core 32-bit LX6 CPU with up to 240 MHz, 520 kB SRAM and 4 MB SPI flash; integrated Wi-Fi 802.11 b/g/n with up to 150 $\frac{\text{Mbit}}{\text{s}}$ and Bluetooth; on-chip 12-bit SAR ADC (analog-to-digital converter) and DAC (digital-to-analog converter). Supplied at 3.0 V to 3.6 V, operating range -40°C to 85°C [7].
- DHT11** Single-wire digital temperature/humidity sensor; range -20°C to 60°C at $\pm 2^{\circ}\text{C}$ and 5 % to 98 % r.H. at $\pm 5\%$ r.H.; sampling at 0.5 Hz [8].
- Joy-it KY-026 flame sensor** Detects flames in the IR band 760 nm to 1100 nm with a 60° detection cone up to 100 cm. Provides an inverted analog output (sampled by the ESP32 ADC) and a potentiometer-adjustable digital threshold output for event-driven triggering [9].
- Joy-it KY-028 thermistor** NTC temperature sensor covering -55°C to 125°C at $\pm 0.5^{\circ}\text{C}$. Exposes an analog reading (ADC-limited rate) and a digital over-temperature flag with adjustable threshold [10].
- TSL2561 light sensor** Digital I^2C ambient-light sensor with two photodiodes (visible and IR); 0.1 lx to 40 000 lx at 0.1 lx resolution [11].

Bosch BME680 Low-power $\text{I}^2\text{C}/\text{SPI}$ sensor combining temperature ($\pm 1.0^{\circ}\text{C}$), humidity ($\pm 3\%$ r.H.), pressure ($\pm 0.6\text{ hPa}$) and a VOC gas channel (2 % to 5 % accuracy for ethane, isoprene, ethanol, acetone, CO) exposed as an IAQ index. Configurable update rate up to 1 Hz; operating range -40°C to 85°C and 300 hPa to 1100 hPa [12].

Auxiliary switches Two switches reserved for future work on AI-based outlier detection with gathered training data (see Section VIII): one marks a reading as a labeled training sample, the other flags a measurement as an outlier.

The hardware design of the sensor node is presented from three complementary perspectives. Figure 1 shows the logical circuit diagram made with the KiCAD software [13] and documents how the ESP32-DevKitC is wired to the sensors, the two auxiliary switches and the power supply, including the GPIO assignment of each signal line. Figure 2 in turn shows the resulting printed circuit board (PCB) layout with the physical placement of the components, the routed copper traces and the board outline intended for fabrication. Both views are exported from the same KiCAD project and together relate the firmware-level GPIO numbers used in the measurement workflow to the corresponding physical pads on the planned PCB. The PCB has not been manufactured yet, so the actual sensor node shown in Figure 3 is built on a prototyping board with the components manually placed and wired according to the circuit diagram.

Measurement Workflow

The firmware on the ESP32 follows a deterministic, two-phase lifecycle written in Arduino C++ [14]: a one-time *startup* phase that prepares the networking and sensing subsystems, followed by a continuously running *measurement loop*.

During startup the node performs the following steps in order:

- 1) Wi-Fi association: The node connects to a configured local network using a pre-provisioned SSID and password.
- 2) Time synchronization: The internal clock is synchronized with an NTP server. An accurate wall-clock time is required both for meaningful measurement timestamps and for the validation of TLS certificates during the subsequent MQTT handshake.
- 3) Sensor initialization: GPIO pin modes are set and sensor-specific configuration (e.g. DHT11 configuration) is applied.
- 4) MQTT setup: The MQTT client is configured (see Section IV-A).

Once startup is complete, the node enters its measurement loop:

- The MQTT connection is continuously validated and kept alive; on disconnect, the client transparently reconnects.
- Every t milliseconds a full sensor reading is taken across all attached sensors with a default waiting period of $t = 1000\text{ ms}$. Values smaller than one second might not change much, as the sensors often have a minimum sampling rate of 1 Hz or lower.

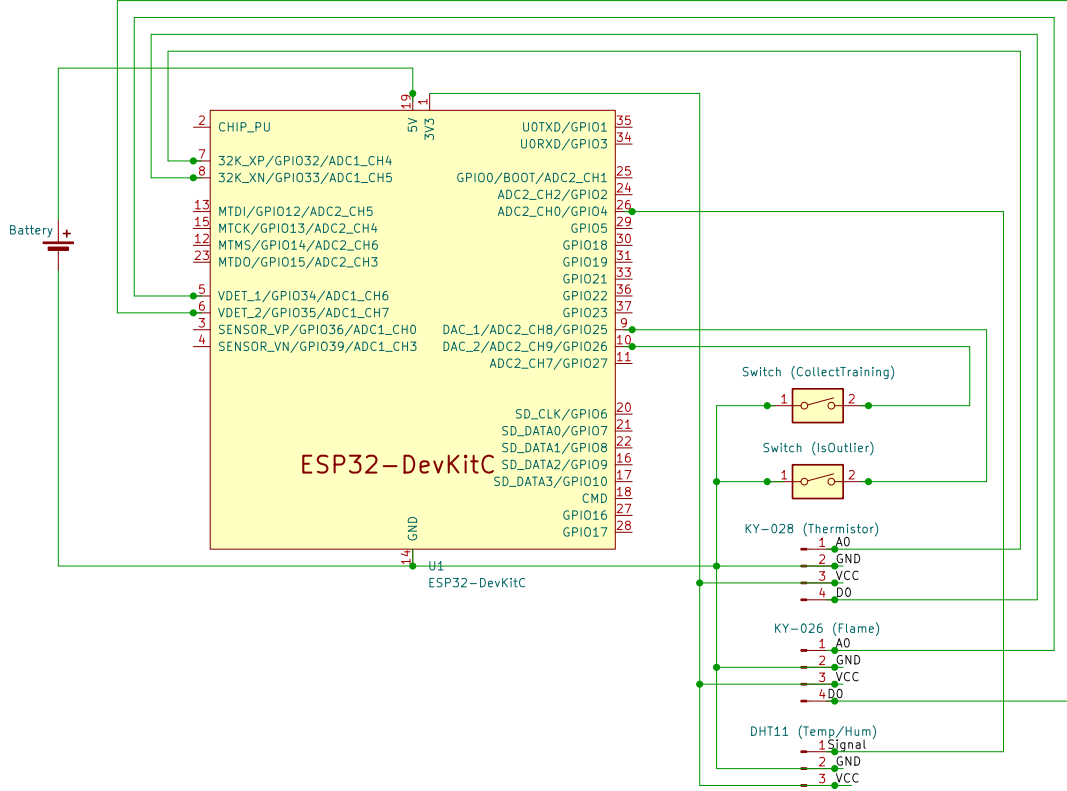


Figure 1. Circuit diagram of the sensor node, showing the logical wiring between the ESP32-DevKitC, sensors, the two auxiliary switches and the power supply, together with their respective GPIO assignments.

- After n consecutive readings the buffered samples are aggregated into a single record, serialized as JSON and published to the configured MQTT topic. The standard sample aggregation size is $n = 5$, guaranteeing a balance between responsiveness and noise reduction.

Aggregation reduces sensor noise and the influence of isolated outliers. It is performed per field according to its data type: the most recent timestamp of the buffer is taken as the record timestamp, numerical values (analog readings) are averaged arithmetically, and digital boolean signals are decided by majority voting over the buffered samples.

Thermistor Conversion: The KY-028 module exposes the thermistor as an analog voltage divider that the ESP32 reads through its 12-bit ADC. The raw ADC value $adc \in [0, 4095]$ is first converted to a resistance R assuming a sensor-internal $10\text{ k}\Omega$ pull-down resistor and a 3.3 V supply, and then to an absolute temperature using the Steinhart–Hart equation [15]. The equation models the resistance–temperature relationship of a semiconductor thermistor as

$$\frac{1}{T} = A + B \ln R + C(\ln R)^3, \quad (1)$$

where T is the temperature in kelvin, R is the thermistor resistance in ohms at temperature T , and A , B and C are

the Steinhart–Hart coefficients characteristic of the specific thermistor over the temperature range of interest. The firmware uses the coefficients $A = 1.129148 \cdot 10^{-3}$, $B = 2.34125 \cdot 10^{-4}$ and $C = 8.76741 \cdot 10^{-8}$, then converts T from kelvin to degrees Celsius before further using the value as `thermistor_temp`.

A. MQTT Publication to AWS IoT Core

The sensor node communicates with AWS IoT Core via the MQTT protocol over TLS.

To establish a secure connection to the AWS IoT broker on port 8883, the node requires the broker address and a set of cryptographic credentials. The publicly available AWS Root CA certificate alongside the individual device certificate and device private key are used for mutual TLS authentication.

The MQTT topic structure is designed around a unique device identifier (e.g., `esp32-lab-001`). Data exchange uses three primary topics: `sensiq/<identifier>/data` for publishing the aggregated sensor records, `sensiq/<identifier>/test` for verifying connectivity, and `sensiq/<identifier>/commands` reserved for receiving subsequent control commands. Listing 1 shows an example of the JSON payload that the ESP32 node publishes to the data topic after each aggregation interval.

V. AWS SERVICES

Detailed description of the cloud-side processing pipeline. Two functional routes are exposed by the backend: a low-

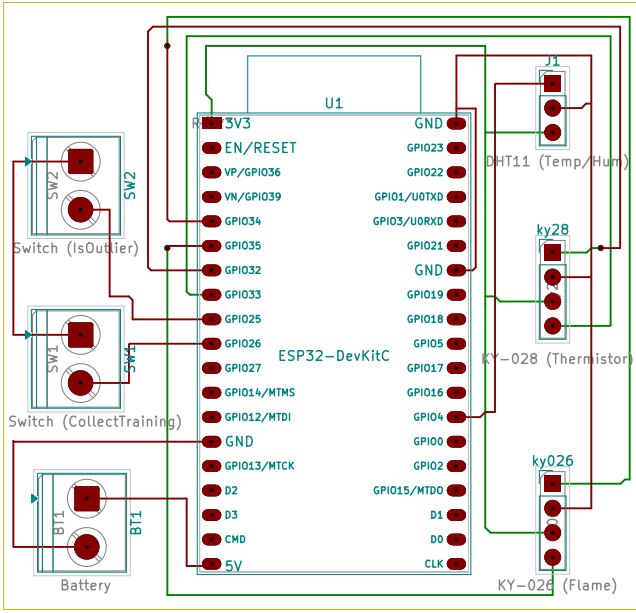


Figure 2. PCB layout derived from the circuit diagram, showing the physical placement of the ESP32-DevKitC and the attached sensors, the routed copper traces and the board outline intended for fabrication.

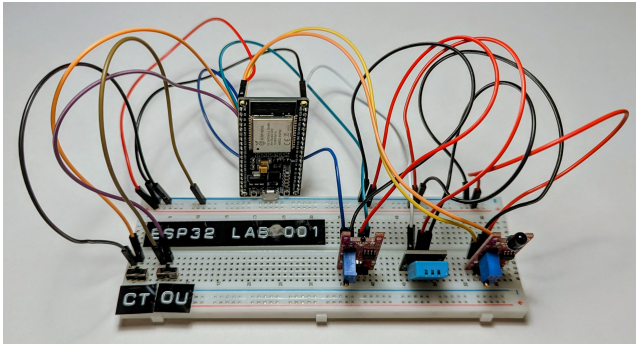


Figure 3. Real physical sensor node with the assembled sensors, the ESP32-DevKitC and the auxiliary switches.

latency live route and a long-term history route. Each is described in its own subsection together with the AWS services it relies on.

A. Live Route

Path from IoT Core rule -> validation Lambda -> DynamoDB (latest reading per device) -> API Gateway -> GET /live. Discuss latency targets, payload shape, and error handling.

B. History Route

Path from IoT Core / validation Lambda -> S3 (Parquet, partitioned by date) -> Glue catalogue -> Athena -> history Lambda -> API Gateway -> GET /history. Discuss partitioning strategy, query patterns, and expected query latency.

VI. USER INTERACTION

Describes how end users consume the data and receive alerts.

```

1 {
2   "running_time": 9881892,
3   "timestamp": "2026-05-25T22:56:12Z",
4   "device_id": "esp32-lab-001",
5   "location": "Lab A, OTH Amberg-Weiden, 92224 Amberg,
6     Germany",
7   "dht_humidity": 61,
8   "dht_temperature": 23.8,
9   "dht_heat_index": 23.82809,
10  "flame_analog": 0,
11  "flame_digital": false,
12  "thermistor_analog": 2005,
13  "thermistor_digital": false,
14  "thermistor_temp": 24.05634,
15  "is_outlier": false,
16  "collect_training": false
17 }
```

Listing 1. Published JSON payload by the ESP32 sensor node via MQTT.

A. Email Notifications via SNS

Explain the alert pipeline: threshold evaluation in the validation Lambda, publication to an SNS topic, and email delivery to subscribed recipients. Cover debounce behaviour and message content.

B. Web Frontend

Describe the React-based dashboard hosted on AWS Amplify: live tiles, historical charts, sensor status indicators, and authentication via AWS Cognito. Include a screenshot or mock-up (see template: figure*).

VII. EVALUATION

A. Testability

1) *Sensor Node Firmware*: The testability of the sensor node, built around the ESP32 microcontroller (see Section IV), presents specific challenges due to its embedded nature. Unit testing of the firmware is implemented using the AUnit framework [18], which is structured such that either the test suite or the main measurement loop executes, preventing interference between testing and regular operation.

Unit tests are implemented for isolated modules such as the sensor readings, sensor data aggregation, Wi-Fi connection, basic MQTT connection and configuration settings. However, exhaustively testing the MQTT publication functionality described in Section IV-A via unit tests is not feasible without extensive mocking. Since such mocking would deviate significantly from real-world behavior, and the underlying MQTT library is already well-tested and established in the community, complex local test implementations for MQTT were omitted.

Instead, the hardware node employs a direct feedback mechanism for connectivity validation during its real-world measurement workflow. Upon successfully establishing a connection to the MQTT broker, the node subscribes to the test

topic to verify the connection's health, ensuring the publish functionality operates correctly in the physical environment.

B. Cost Estimation

1) *Hardware Cost*: The one-time hardware cost per sensor node is based on the components used for prototyping. A breakdown of the individual component costs is presented in Table I.

Table I
HARDWARE COST PER SENSOR NODE

Component	Unit Cost
ESP32-DevKitC	8 €
Bosch BME680	18 €
TSL2561 Light Sensor	5 €
DHT11 Sensor	3 €
Joy-it KY-026 Flame Sensor	3 €
Joy-it KY-028 Thermistor	3 €
Prototyping equipment	~3 €
Total	~43 €

VIII. SUMMARY AND FUTURE WORK

Recap the contribution of the report: a working, MCU-based IoT architecture for environmental monitoring with secure ingestion, dual-path persistence, and a user-facing dashboard. Outline next steps: extended sensor coverage, predictive analytics on historical data, multi-tenant support, and hardening for production deployment.

Future work notes: - Deep Sleep for ESP32 on Hardware for longer battery life - AI Outlier detection on AWS - Predictions on future sensor readings

REFERENCES

- [1] Claes Ahlneck and George Zografi. "The molecular basis of moisture effects on the physical and chemical stability of drugs in the solid state". In: *International journal of pharmaceuticals* 62.2-3 (1990), pp. 87–95.
- [2] Norbert Kockmann, Philipp Thenée, Christoph Fleischer-Trebes, Gabriele Laudadio, and Timothy Noël. "Safety assessment in development and operation of modular continuous-flow processes". In: *Reaction Chemistry & Engineering* 2.3 (2017), pp. 258–280.
- [3] Rafia Mumtaz, Syed Mohammad Hassan Zaidi, Muhammad Zeeshan Shakir, Uferah Shafi, Muhammad Moez Malik, Ayesha Haque, Sadaf Mumtaz, and Syed Ali Raza Zaidi. "Internet of things (Iot) based indoor air quality sensing and predictive analytic—A COVID-19 perspective". In: *Electronics* 10.2 (2021), p. 184.
- [4] Gonalo Marques and Rui Pitarma. "An internet of things-based environmental quality management system to supervise the indoor laboratory conditions". In: *Applied Sciences* 9.3 (2019), p. 438.
- [5] Simon Völkl, Johannes Schieder, Sebastian Rosner, and Andre Tien Vu. *Sensiq GitHub Repository*. [Online]. URL: <https://github.com/s-voelkl/BCN-2026SSTeamMagenta-Sensiq>.
- [6] BerryBase. *ESP32 NodeMCU Development Board*. [Online]. URL: <https://www.berrybase.de/en/esp32-nodemcu-development-board>.

- [7] Espressif Systems. *ESP32-WROOM-32 Datasheet*. [Online]. URL: https://documentation.espressif.com/esp32-wroom-32-datasheet_en.pdf.
- [8] Asair. *DHT11 Temperature and Humidity Module*. [Online]. URL: <https://asairsensors.com/product/dht11-sensor/>.
- [9] Joy-It. *KY-026 Analog Flame Sensor*. [Online]. URL: <https://sensorkit.joy-it.net/en/sensors/ky-026>.
- [10] Joy-It. *KY-028 Temperature Sensor (Thermistor)*. [Online]. URL: <https://sensorkit.joy-it.net/en/sensors/ky-028>.
- [11] BerryBase. *TSL2561 Digital Light / Brightness Sensor Datasheet*. [Online]. URL: <https://www.berrybase.de/en/product-datasheet/019234a424aa7020aee21ad30f4ed8c8/create>.
- [12] Bosch Sensortec. *BME680 Gas, Pressure, Temperature and Humidity Sensor Datasheet*. [Online]. URL: <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme680-ds001.pdf>.
- [13] KiCad Services Corporation. *KiCad PCB Design Suite*. Version 10.0.3. 2026. URL: <https://www.kicad.org>.
- [14] Arduino. *Arduino Language Reference*. [Online]. URL: <https://www.arduino.cc/reference/en/>.
- [15] John S Steinhart and Stanley R Hart. "Calibration curves for thermistors". In: *Deep sea research and oceanographic abstracts*. Vol. 15. 4. Elsevier. 1968, pp. 497–503.
- [18] Brian T. Park. *AUnit Testing Framework for Arduino*. Version 1.7.1. 2026. URL: <https://github.com/bxparks/AUnit>.